

Linear Data Structures

✓ Arrays , ✓ Link Lists, ✓ Stacks and ✓ Queues



Deliverables

Arrays

Link Lists

Stacks

Queues



Arrays



Arrays are stored in contiguous memory locations and contain similar data. Position of any array element can be accessed from its index position with help of array's starting address.

An element can be accessed, inserted or removed by specifying its position (number of elements preceding it)

Vector is abstraction of array data structure, where we can access by rank.

Printing/Accessing — Insertion — Deletion — Array operations

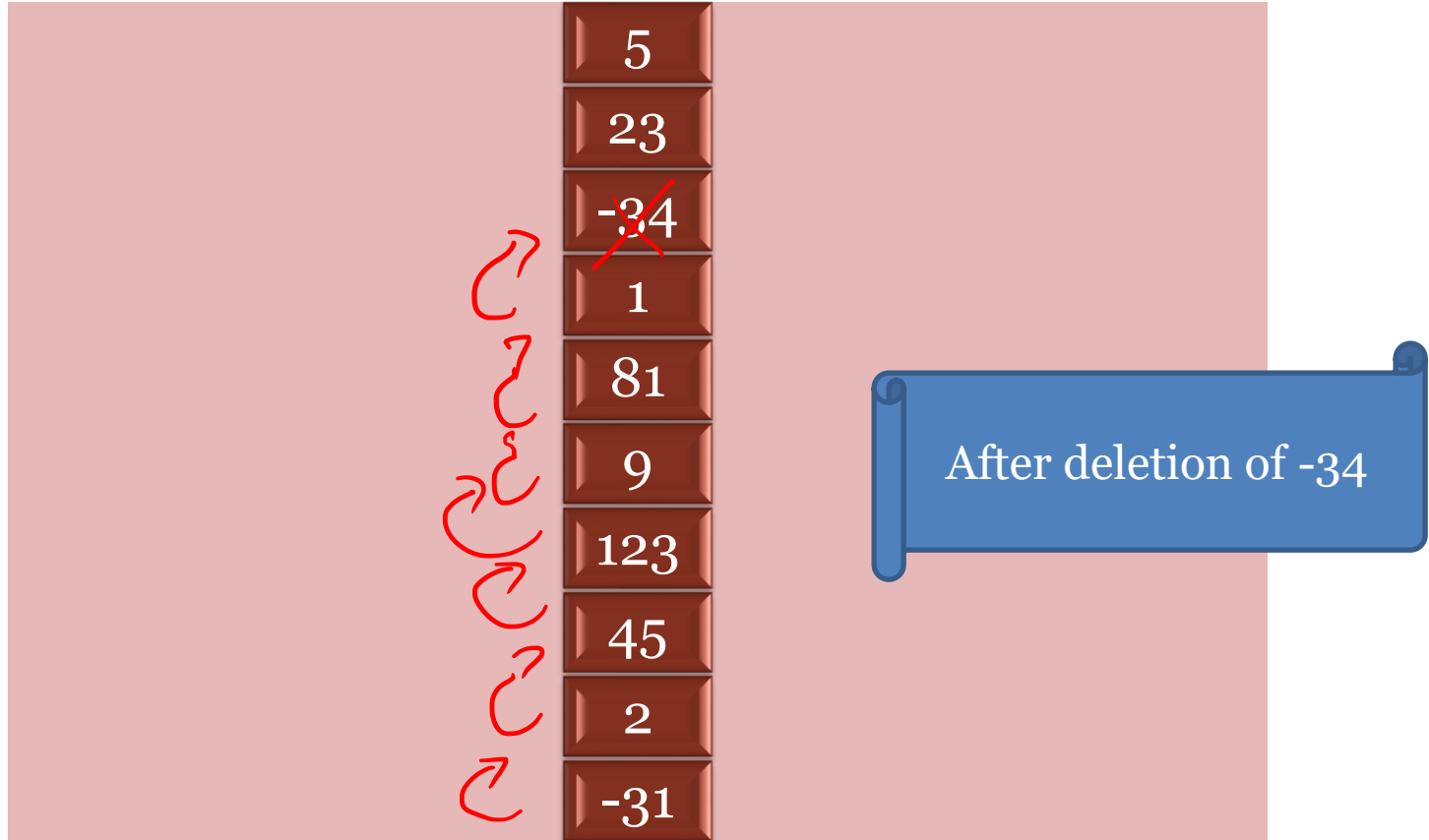
- Deletion: In operation removeAtPosition(r), we need to fill the hole left by the removed element by shifting backward $n-r-1$ elements $A[r + 1]$, ..., $A[n - 1]$. In the worst case ($r = 0$), this takes $O(n)$ time. In the average case also it is $O(n)$.

- $A[0]$ $A[1]$ $A[2]$ $A[3]$ $A[4]$ $A[5]$ $A[6]$ $A[7]$ $A[8]$ $A[9]$

3	10	0	-135	2	21	41	6	87	11
---	----	---	------	---	----	----	---	----	----

- Is it necessary to shift the array elements after deletion?

Deletion of -34



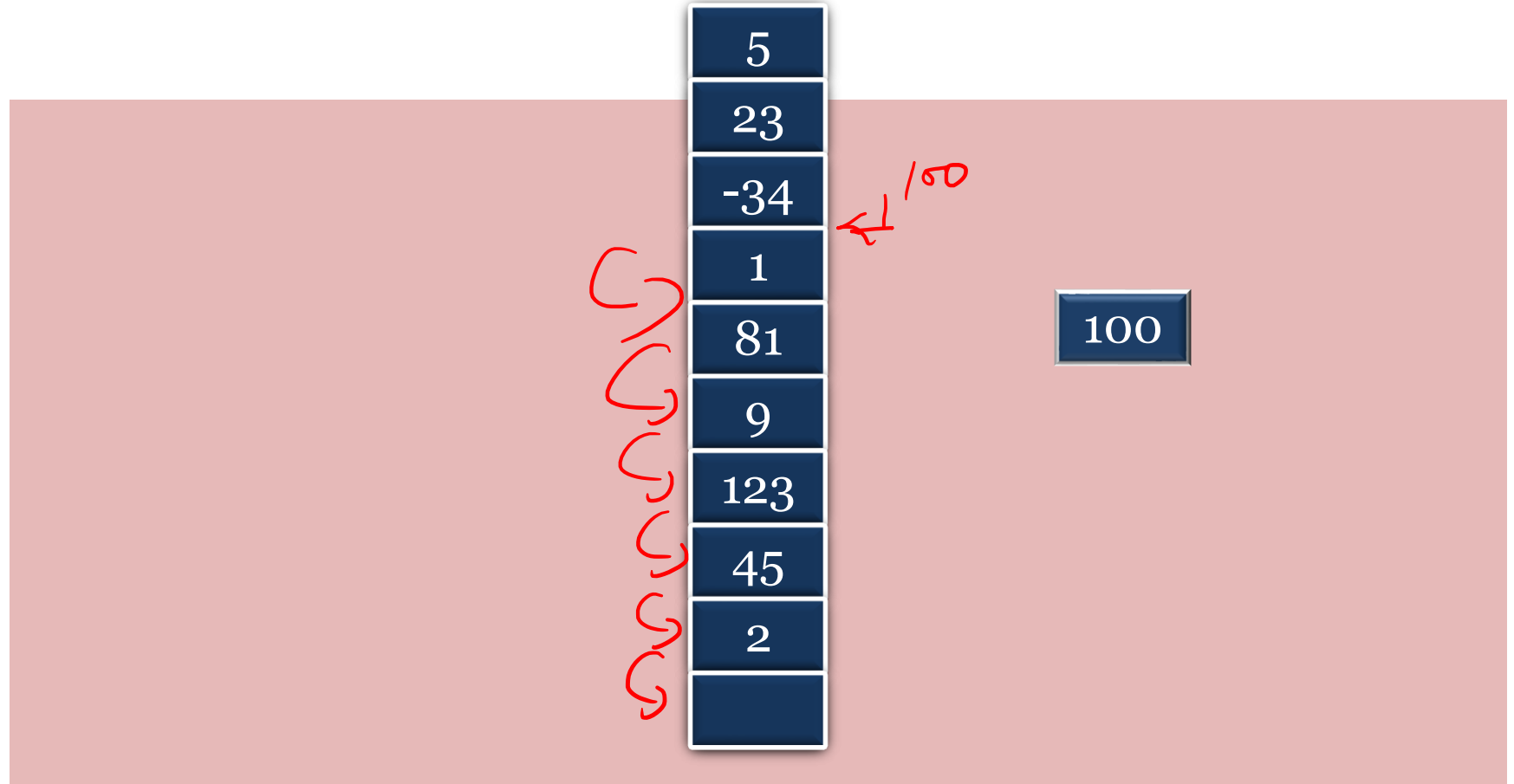
Insertion

Insertion in the array again requires shifting all the elements one index ahead to make room for insertion.

Need to understand shifting different in deletion and insertion, otherwise data may be lost.

In Insertion shifting is to start from the last element of the array and in deletion shifting is to start from the next array index from where the element is to be deleted.

Inserting 100 after -34



Array complexity

- ✓ Accessing an array: $O(1)$ time
- ✓ Copying an array: $O(\underline{n})$ time
- ✓ Merging two arrays: $O(\underline{m+n})$ time
- ✓ Splitting an array: $O(\underline{n})$ time
- ✓ Intersection of two arrays: $O(\underline{n^2})$ ✓ ?
- Union of two arrays: $O(n^2)$ ✓

look for each element
in another array.

Array incrementing by C or doubling

Let n be the number of elements in the array. The space used is $O(n)$ and each operation runs in time $O(1)$

The maximum size of the array must be defined apriori and cannot be changed.

✓ Incremental strategy: increase the size by a constant C → $O(n^2)$

✓ Doubling strategy: double the size → $O(n)$

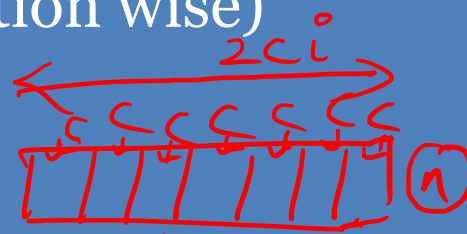
Array incrementing by C or Doubling

If increment by C every time then

Creation copying insertion (iteration wise)

c_i $c(i-1)$ c (new one)

Total = c_i + c_i - c + c = $2c_i$ ✓



$2c(1 + 2 + \dots + \frac{n}{c})$ $\frac{n^2}{c}$

If there are n elements in end then total iterations will be n/c , So it is $2c(1+2+3+\dots+n/c) =$

$\frac{2c(\frac{n}{c})(\frac{n}{c} + 1)}{2} = \frac{n^2}{c} = O(n^2)$

Array incrementing by C or Doubling

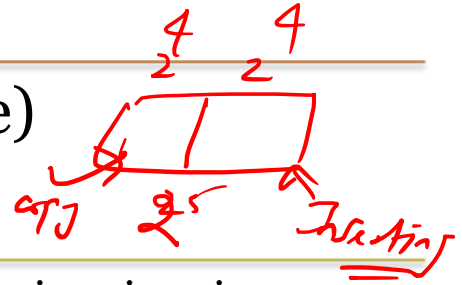
If we double the array every time then the creation, copying, insertion cost will be as follows

Creation copying insertion (iteration wise)

$$2^i$$

$$2^{i-1}$$

$$2^{i-1}$$



Total cost in every iteration is $2^i + (2^{i-1} + 2^{i-1}) = 2^i + 2^i = 2^{i+1}$

$$2^L = 1 \Rightarrow L = \log n$$

if there are n elements to be inserted then there will be $\log n$ iterations, total becomes $(2^1 + 2^2 + 2^3 + \dots + 2^{\log n + 1}) = 4n - 1 = O(n)$

links

Link List



Singly linked list is a data structure consisting of a sequence of nodes

Each node stores data and link to the next node

There may be a header link and trailer link

Link list operations

Access (): Accessing an element from a list



Worst case: $O(n)$



Average Case: $1+2+3+\dots+n = \frac{n(n-1)}{2}$ divided by $n = \frac{n-1}{2} = O(n)$



Delete (): $O(1)$ complexity, but it is preceded by accessing that position, effectively taking $O(n)$ time. Delete first takes $O(1)$ time



Link list complexity

Insert : $O(1)$ but inserting before or after an element will require accessing element effectively taking $O(n)$ time. Inserting at first place is $O(1)$.

Merging : $O(1)$ If the trailer node information is not available then it will be equal to the length of the smaller list to reach at the end of the list for merging with the second list

Size: $O(n)$ time

Intersection: $O(n^2)$

Union: $O(n^2)$

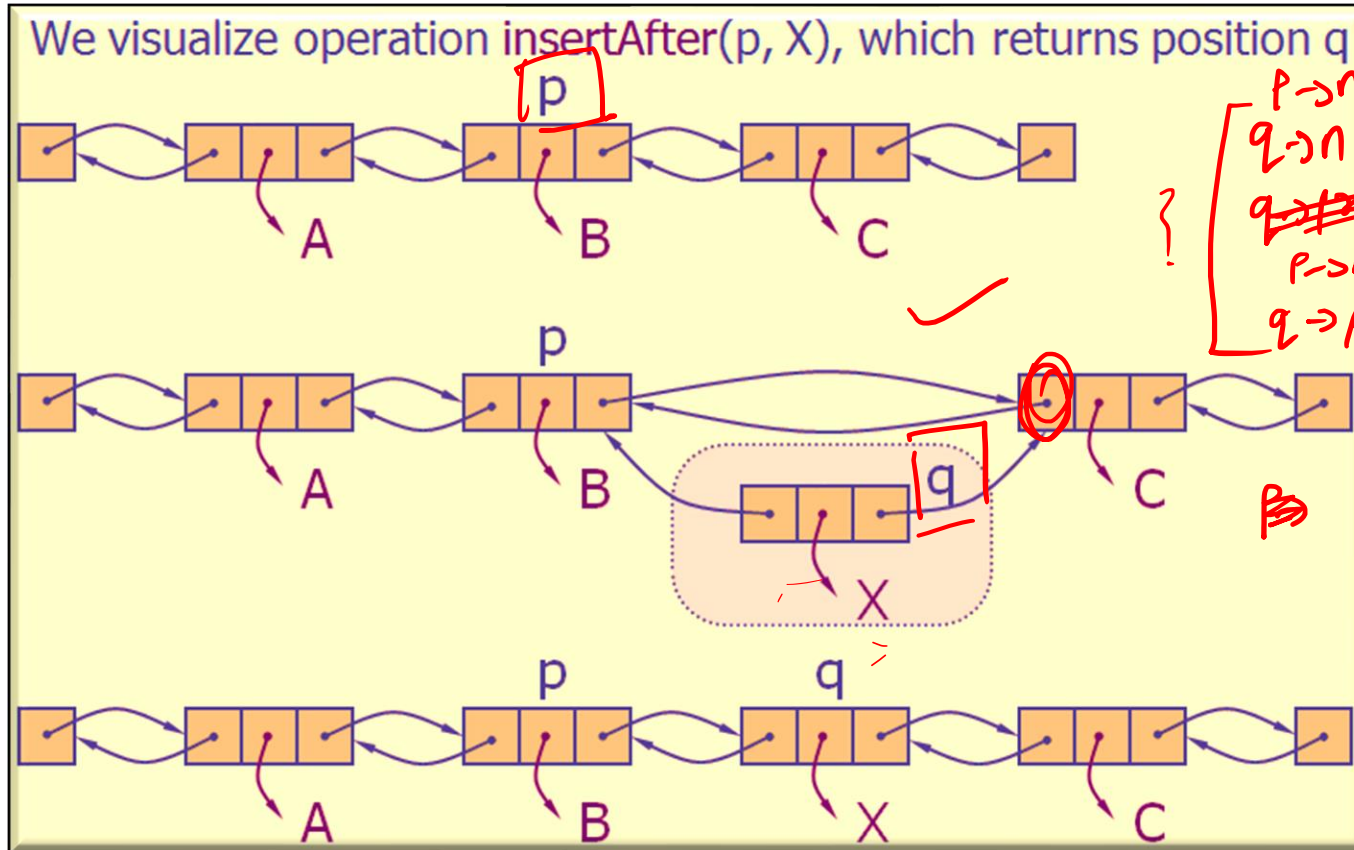
Doubly link list

A doubly linked list provides a natural implementation of the List

Nodes implement Position and stores element, link to the previous node and link to the next node

Special trailer and header nodes. Deletion and Insertion becomes easy as there is no need to remember the address of the previous node.

Insertion

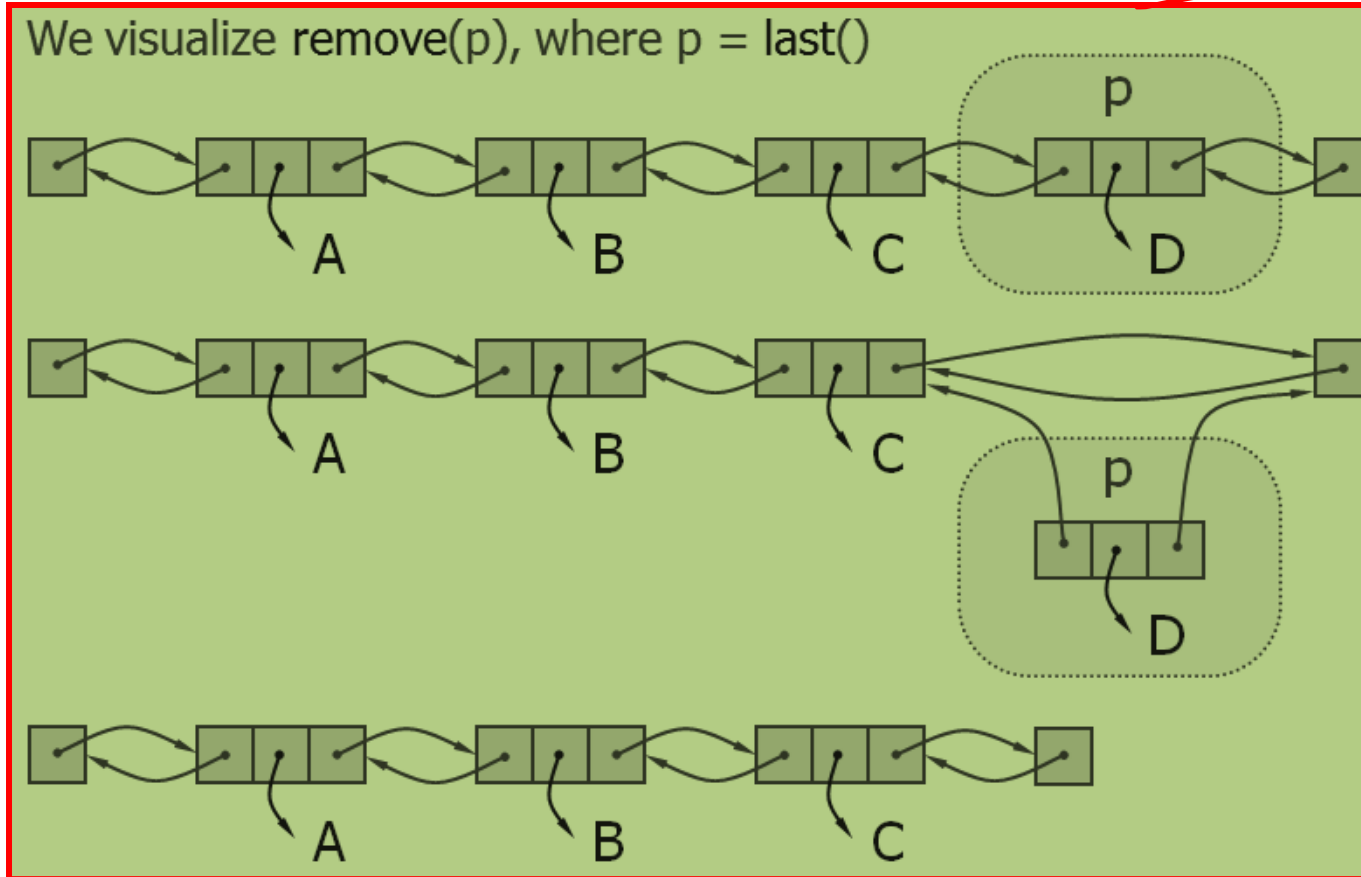


Handwritten notes:

- $p \rightarrow \text{next} \rightarrow (p \rightarrow q)$
- $q \rightarrow \text{next} = p \rightarrow \text{next}$
- $q \rightarrow \text{prev} = p$
- $p \rightarrow \text{next} = q$
- $q \rightarrow \text{prev} = p$

Deletion

$p \rightarrow \text{next} \rightarrow p \rightarrow \text{prev} = p \rightarrow \text{prev};$
 $p \rightarrow \text{prev} \rightarrow \text{next} = p \rightarrow \text{next};$



One pointer ✓

last node $\rightarrow$ next = head

Circular link list



There will be no difference in the time complexity of various operations

Circular Link List and circular doubly link list
Input output buffers use circular queue for buffering.

Comparison link list vs. array

It is easier to delete in the link list

it is easier to insert in the link list

It is easier to access in the array

We have to predefine the array so there can be wastage of memory

no shifting

indexing

wastage

Comparison link list vs. array

Array Size has to be defined and is limited to the defined size while link list can grow to limit of memory

Continuous allocation is required for array, while this is not with link list

Arrays compute address of data elements by arithmetic operations while in linked list addresses are in structure itself.

Arrays can be accessed backward and forward but we have to use special link lists like doubly linked list for backward access

Comparison link list vs. array

Arrays definition is *implemented already.* part of the language construct but link list we have to create.

Merging two arrays is very difficult while merging link lists is easy

There will be more cache misses in the Link list then in the arrays

With large records, *moving pointers* is easier and faster than moving the items themselves.

Comparison link list vs. array

There can be an array of pointers associated with a link list, in that case address part of the link list is not required.

In link lists, two different processors may work on two different parts of link list.

Due to address part, there is wastage of memory in link list, it is not much if we look at practical size of record of name, age etc.

bigger data

Stack



(LIFO)

Last in First out,
in and out only
from one end,
other end is
closed

Page-visited
history in a Web
browser

Undo sequence
in a text editor

Chain of method
calls in the Java
Virtual Machine,
Function Calls

Recursion

In other data
Structures



Simulating Pop and Push

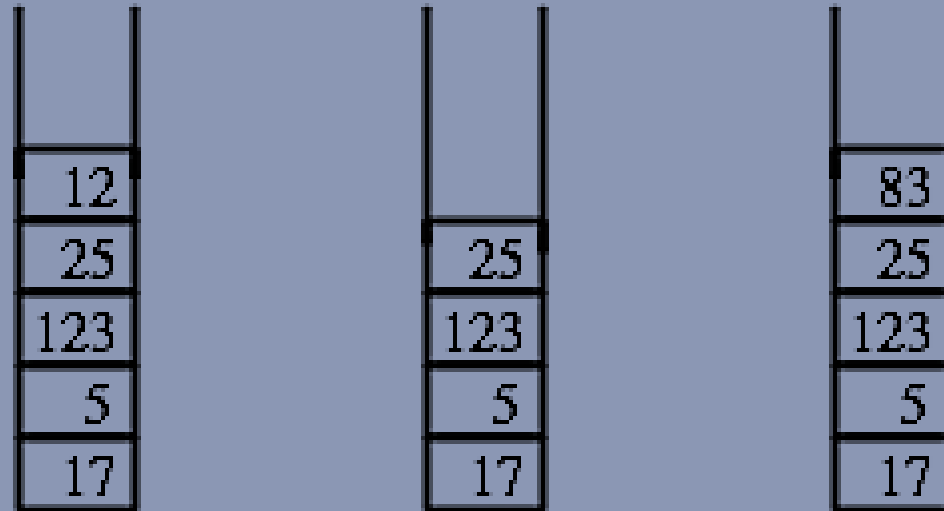
Apply Operations

1. Pop
2. Pop
3. Push 37



37

Stack Example



Original stack. After pop(). After push(83).

Stack

✓ Creation of an stack: $O(n)$ ✓

push(object): inserts an element $O(1)$ ✓

pop(): removes and returns the last inserted element. $O(1)$ ✓

top(): returns last inserted element without removing $O(1)$ ✓

size(): returns number of elements stored. $O(1)$ ✓

isEmpty(): indicates whether no elements are stored. $O(1)$ ✓

isFull(): Indicates whether array limit is over. $O(1)$ ✓

Link List Vs. Array Implementation

(Stack)

Pop

Array: return element at “top” and decrement “top”

Linked list: return and remove at front of linked list

Push

Array: increment “top” and insert element. Must check for overflow!

Linked list: insert element at front of linked list

Array: more memory efficient

Linked list: don't have to worry about “overflow”



Axioms

$$\text{pop}(\text{push}(S, v)) = S$$

$$\text{Top}(\text{push}(S, v)) = v$$

$$\text{Top}(\text{pop}(\text{push}(\text{push}(S, 12), 15))) = \text{top}(\text{push}(S, 12))$$

Queues



Insertions and deletions follow the first-in first-out scheme

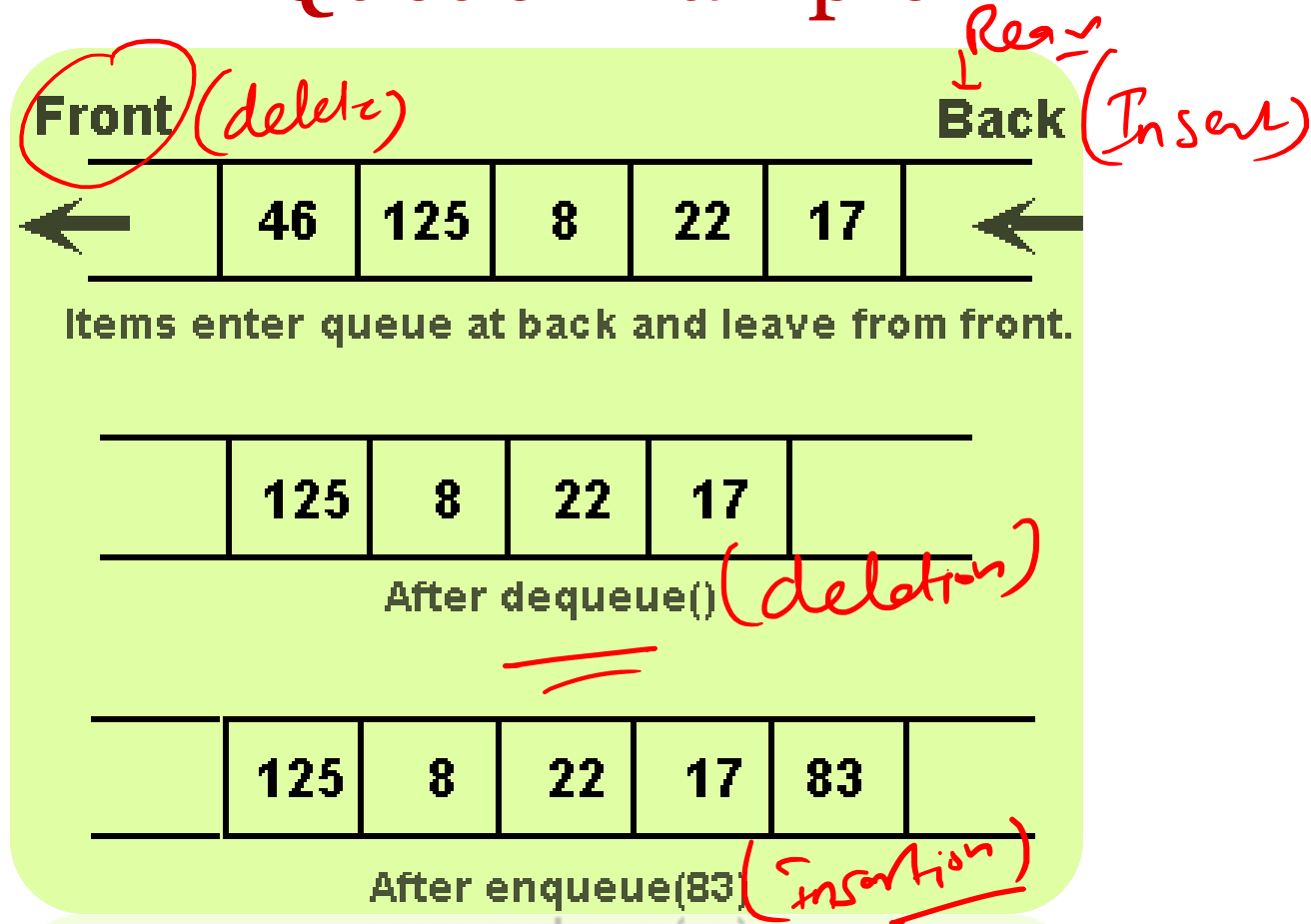
FIFO

Insertions are at the rear of the queue and removals are at the front of the queue

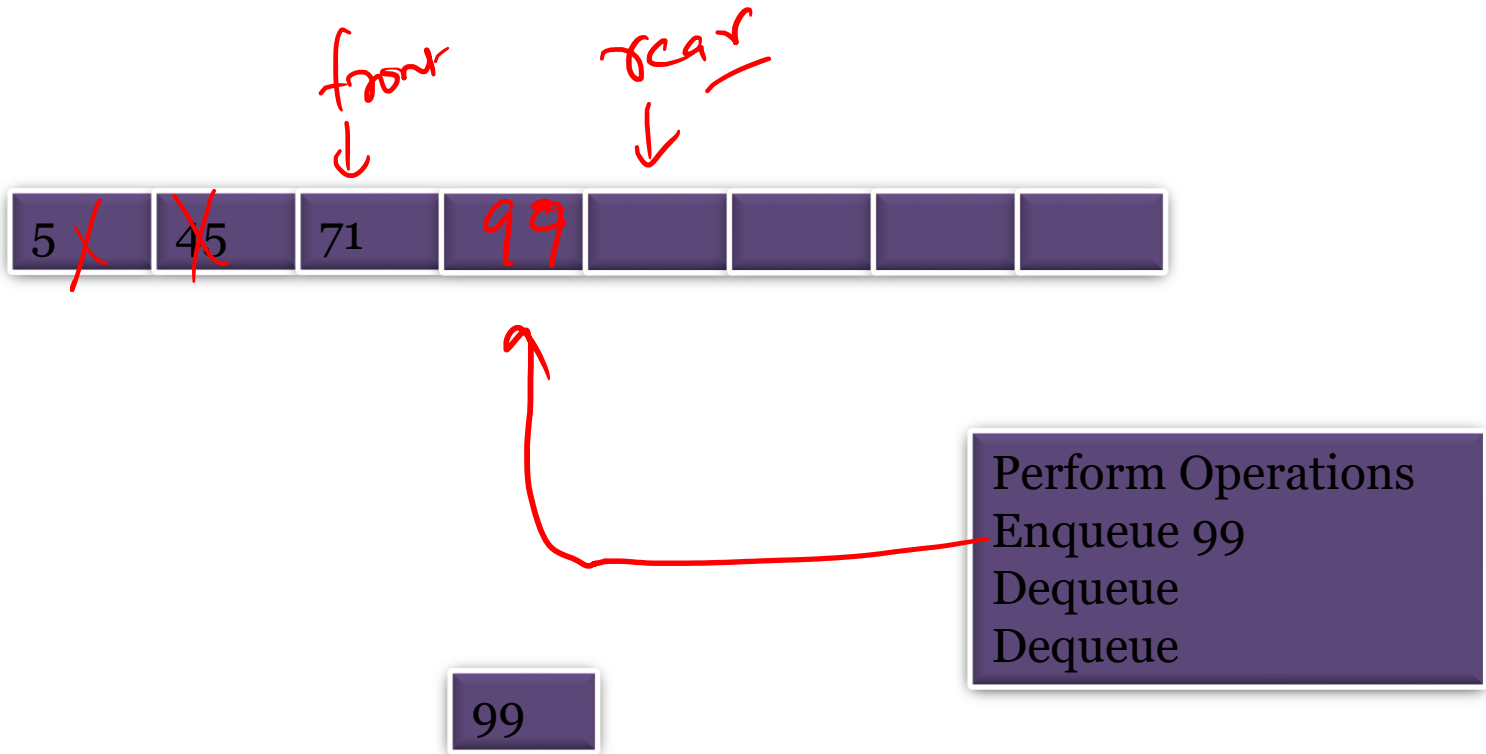
Two variables keep track of the front and rear. Careful implementation is necessary to avoid overlapping.

Operations complexity may vary depending upon the implementation, whether front or rear is fixed or not

Queue Example



Simulating Enqueue and Dequeue



Queues

`enqueue(Object o):`
inserts an
element `o` at end
of queue. $O(1)$

`dequeue():` removes
and returns front
element of
queue. $O(1)$

`front():` returns
element at front
without
removing it. $O(1)$

`size():` returns
number of
elements stored.
 $O(1)$

`isEmpty():` return
s Boolean
indicating queue
is empty or not
 $O(1)$

Queue Applications

Waiting lists, bureaucracy

Access to shared resources (e.g. printer)

Multiprogramming

Tunnel

In other Data Structures

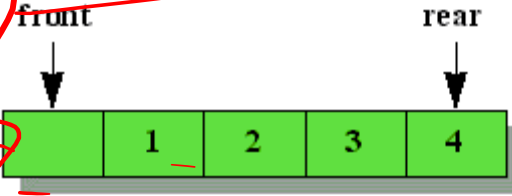
Axioms

$$\text{Front}(\text{Enqueue}(\text{Enqueue}(Q, w), v)) = \text{front}(\text{Enqueue}(Q, w))$$
$$\text{Deque}(\text{Enque}(\text{Enque}(Q, w), v)) = \text{Enque}(\text{Deque}(\text{Enque}(Q, w)), v)$$

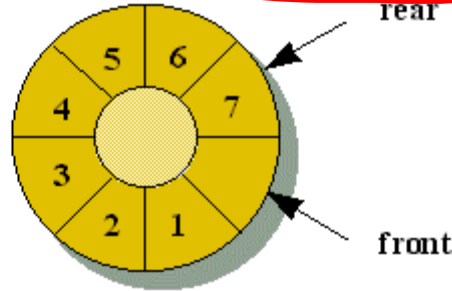
In an Enqueue operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one, Similar to what we did for an array based stack.

To fill the gap
of queue
without shifting

Circular queue

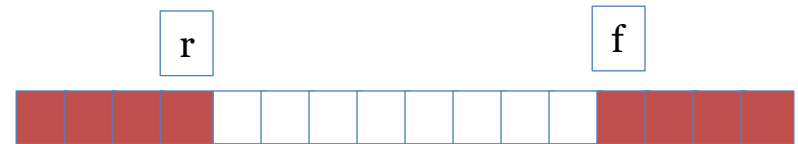


linear queue



circular queue

Circular Queue is essentially implemented as a linear array in a wrapped around fashion where even rear pointer of the queue can come before the front pointer



Questions, Comments and Suggestions



Question 1

What is result of Last operation in given sequence:

Push(&);

Push(\$);

Push(@);

Top();

Pop();

Push(#);

Pop();

Pop();

Push(%);

Pop();

Question 2

What will be the result of last result of given priority queue if number 1 is the highest priority

```
Enqueue(3)
Enqueue(6)
Enqueue(2)
Dequeue
Dequeue
Enqueue(5)
Enqueue(1)
dequeue
dequeue
```

Question 3

In a Singly circular linked list organization, insertion of a record involves modification of

- A. One pointer
- ☒ B. Two pointers
- C. Three pointers
- D. Four pointer
- E. No Pointer

